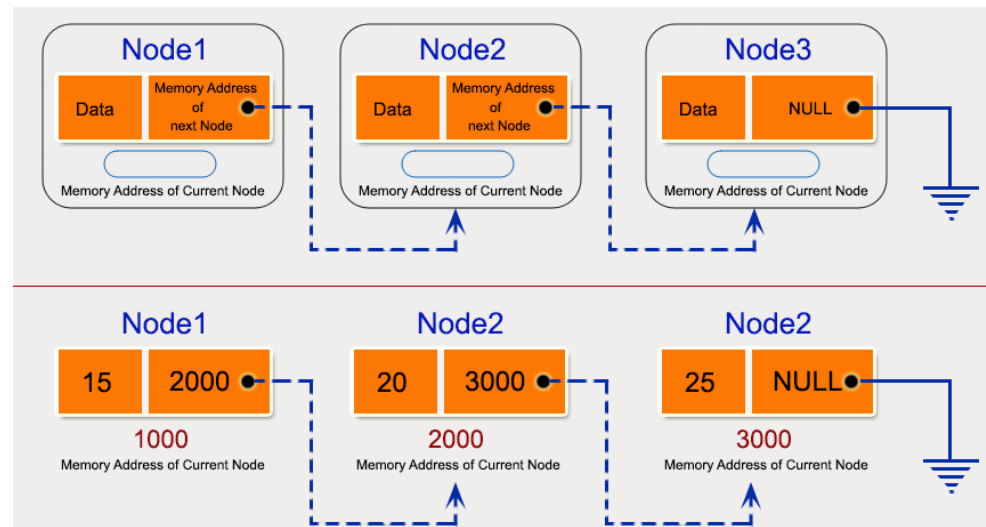


1. Linked List Creation Tricks

- Write a program in C to create and display a Singly Linked List.
- Test Data :
- Input the number of nodes : 3
Input data for node 1 : 5
Input data for node 2 : 6
Input data for node 3 : 7
- *Expected Output :*
- Data entered in the list :
- Data = 5
- Data = 6
- Data = 7



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Structure for a node in a linked list
5  struct node {
6      int num;           // Data of the node
7      struct node *nextptr; // Address of the next node
8  } *stnode;             // Pointer to the starting node
9
10 // Function prototypes
11 void createNodeList(int n); // Function to create the linked list
12 void displayList();         // Function to display the linked list
13
```

```
14 // Main function
15 int main() {
16     int n;
17
18     // Displaying the purpose of the program
19     printf("\n\n Linked List : To create and display Singly Linked List :\n");
20     printf("-----\n");
21
22     // Inputting the number of nodes for the linked list
23     printf(" Input the number of nodes : ");
24     scanf("%d", &n);
25
26     // Creating the linked list with n nodes
27     createNodeList(n);
28
29     // Displaying the data entered in the linked list
30     printf("\n Data entered in the list : \n");
31     displayList();
32
33     return 0;
34 }
35
```

```
36 // Function to create a linked list with n nodes
37 void createNodeList(int n) {
38     struct node *fnNode, *tmp;
39     int num, i;
40
41     // Allocating memory for the starting node
42     stnode = (struct node *)malloc(sizeof(struct node));
43
44     // Checking if memory allocation is successful
45     if(stnode == NULL) {
46         printf(" Memory can not be allocated.");
47     } else {
48         // Reading data for the starting node from user input
49         printf(" Input data for node 1 : ");
50         scanf("%d", &num);
51         stnode->num = num;
52         stnode->nextptr = NULL; // Setting the next pointer to NULL
53         tmp = stnode;
54
```

```
55 // Creating n nodes and adding them to the linked list
56 for(i = 2; i <= n; i++) {
57     fnNode = (struct node *)malloc(sizeof(struct node));
58
59     // Checking if memory allocation is successful
60     if(fnNode == NULL) {
61         printf(" Memory can not be allocated.");
62         break;
63     } else {
64         // Reading data for each node from user input
65         printf(" Input data for node %d : ", i);
66         scanf(" %d", &num);
67
68         fnNode->num = num; // Setting the data for fnNode
69         fnNode->nextptr = NULL; // Setting the next pointer to NULL
70
71         tmp->nextptr = fnNode; // Linking the current node to fnNode
72         tmp = tmp->nextptr; // Moving tmp to the next node
73     }
74 }
75 }
76 }
77
```

```
78 // Function to display the linked list
79 void displayList() {
80     struct node *tmp;
81
82     if(stnode == NULL) {
83         printf(" List is empty.");
84     } else {
85         tmp = stnode;
86
87         // Traversing the linked list and printing each node's data
88         while(tmp != NULL) {
89             printf(" Data = %d\n", tmp->num); // Printing the data of the current node
90             tmp = tmp->nextptr;               // Moving to the next node in the list
91         }
92     }
93 }
```

2. Linked List Reverse Display

- Write a program in C to create a singly linked list of n nodes and display it in reverse order.

- Test Data :

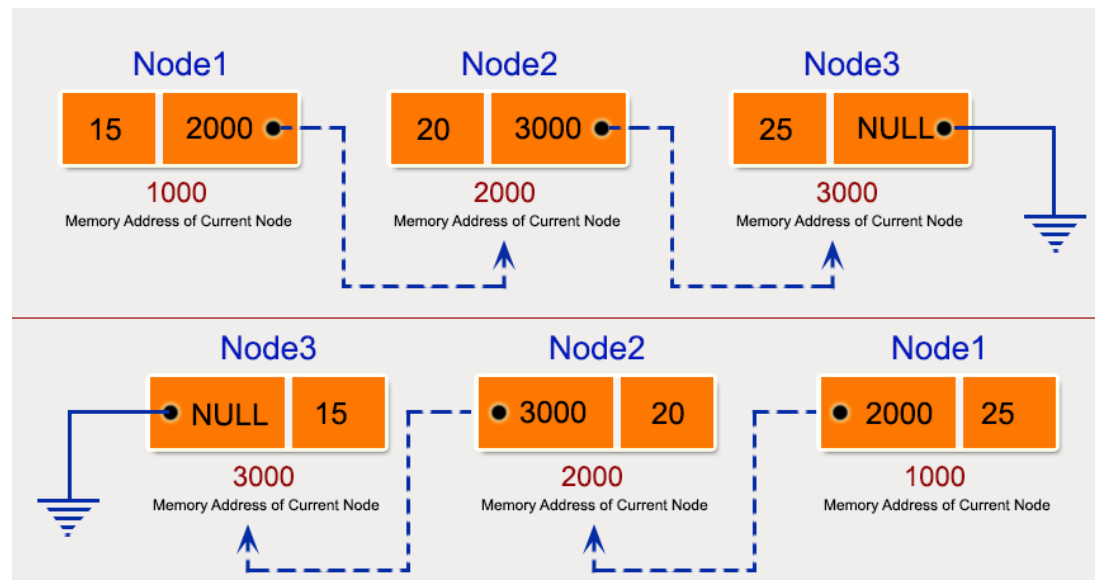
Input the number of nodes : 3

Input data for node 1 : 5

Input data for node 2 : 6

Input data for node 3 : 7

- *Expected Output :*
- Data entered in the list are :
- Data = 5
- Data = 6
- Data = 7
-
- The list in reverse are :
- Data = 7
- Data = 6
- Data = 5



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Structure for a node in a linked list
5  struct node {
6      int num;           // Data of the node
7      struct node *nextptr; // Address of the next node
8  } *stnode;             // Pointer to the starting node
9
10 // Function prototypes
11 void createNodeList(int n); // Function to create the linked list
12 void reverseDispList();     // Function to reverse the linked list
13 void displayList();         // Function to display the linked list
14
```



```
15 // Main function
16 int main() {
17     int n;
18
19     // Displaying the purpose of the program
20     printf("\n\n Linked List : Create a singly linked list and print it in reverse order :\n");
21     printf("-----\n");
22
23     // Inputting the number of nodes for the linked list
24     printf(" Input the number of nodes : ");
25     scanf("%d", &n);
26
27     // Creating the linked list with n nodes
28     createNodeList(n);
29     printf("\n Data entered in the list are : \n");
30
31     // Displaying the data entered in the linked list
32     displayList();
33
34     // Reversing the linked list
35     reverseDispList();
36     printf("\n The list in reverse are :  \n");
37
38     // Displaying the reversed linked list
39     displayList();
40
41     return 0;
42 }
43
```

```
44 // Function to create a linked list with n nodes
45 void createNodeList(int n) {
46     struct node *fnNode, *tmp;
47     int num, i;
48
49     // Allocating memory for the starting node
50     stnode = (struct node *)malloc(sizeof(struct node));
51
52     // Checking if memory allocation is successful
53     if(stnode == NULL) {
54         printf(" Memory can not be allocated.");
55     } else {
56         // Reading data for the starting node from user input
57         printf(" Input data for node 1 : ");
58         scanf("%d", &num);
59         stnode->num = num;
60         stnode->nextptr = NULL; // Setting the next pointer to NULL
61         tmp = stnode;
62
```

```
63     // Creating n nodes and adding them to the linked list
64     for(i = 2; i <= n; i++) {
65         fnNode = (struct node *)malloc(sizeof(struct node));
66
67         // Checking if memory allocation is successful
68         if(fnNode == NULL) {
69             printf(" Memory can not be allocated.");
70             break;
71         } else {
72             // Reading data for each node from user input
73             printf(" Input data for node %d : ", i);
74             scanf(" %d", &num);
75
76             fnNode->num = num;        // Setting the data for fnNode
77             fnNode->nextptr = NULL; // Setting the next pointer to NULL
78
79             tmp->nextptr = fnNode; // Linking the current node to fnNode
80             tmp = tmp->nextptr;    // Moving tmp to the next node
81         }
82     }
83 }
84 }
85
```

```
86 // Function to reverse the linked list
87 void reverseDispList() {
88     struct node *prevNode, *curNode;
89
90     if(stnode != NULL) {
91         prevNode = stnode;
92         curNode = stnode->nextptr;
93         stnode = stnode->nextptr;
94
95         prevNode->nextptr = NULL; // Convert the first node as last
96
97         while(stnode != NULL) {
98             stnode = stnode->nextptr;
99             curNode->nextptr = prevNode;
100
101             prevNode = curNode;
102             curNode = stnode;
103         }
104         stnode = prevNode; // Convert the last node as head
105     }
106 }
107
```

```
108 // Function to display the linked list
109 void displayList() {
110     struct node *tmp;
111     if(stnode == NULL) {
112         printf(" No data found in the list.");
113     } else {
114         tmp = stnode;
115         while(tmp != NULL) {
116             printf(" Data = %d\n", tmp->num);    // Prints the data of current node
117             tmp = tmp->nextptr;                  // Advances the position of current node
118         }
119     }
120 }
```

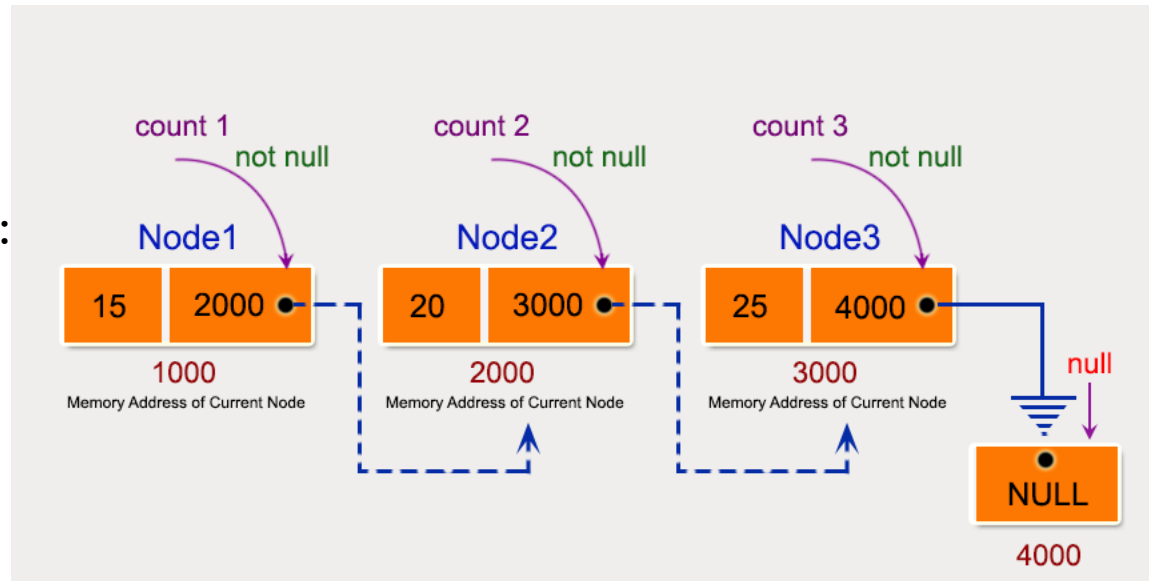
3. Linked List Node Counter Variants

- Write a program in C to create a singly linked list of n nodes and count the number of nodes.
- Test Data :

Input the number of nodes : 3
Input data for node 1 : 5
Input data for node 2 : 6
Input data for node 3 : 7

Expected Output :

- Data entered in the list are :
- Data = 5
- Data = 6
- Data = 7
-
- Total number of nodes = 3



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Structure for a node in a linked list
5  struct node {
6      int num;           // Data of the node
7      struct node *nextptr; // Address of the next node
8  } *stnode;             // Pointer to the starting node
9
10 // Function prototypes
11 void createNodeList(int n); // Function to create the linked list
12 int NodeCount();           // Function to count the nodes
13 void displayList();         // Function to display the linked list
14
```

```
15 // Main function
16 int main() {
17     int n, totalNode;
18
19     // Displaying the purpose of the program
20     printf("\n\n Linked List : Create a singly linked list and count the number of nodes :\n");
21     printf("-----\n");
22
23     // Inputting the number of nodes for the linked list
24     printf(" Input the number of nodes : ");
25     scanf("%d", &n);
26
27     // Creating the linked list with n nodes
28     createNodeList(n);
29     printf("\n Data entered in the list are : \n");
30
31     // Displaying the data entered in the linked list
32     displayList();
33
34     // Counting the number of nodes in the linked list
35     totalNode = NodeCount();
36     printf("\n Total number of nodes = %d\n", totalNode);
37
38     return 0;
39 }
40
```



```
41 // Function to create a linked list with n nodes
42 void createNodeList(int n) {
43     struct node *fnNode, *tmp;
44     int num, i;
45
46     // Allocating memory for the starting node
47     stnode = (struct node *)malloc(sizeof(struct node));
48
49     // Checking if memory allocation is successful
50     if(stnode == NULL) {
51         printf(" Memory can not be allocated.");
52     } else {
53         // Reading data for the starting node from user input
54         printf(" Input data for node 1 : ");
55         scanf("%d", &num);
56         stnode->num = num;
57         stnode->nextptr = NULL; // Setting the next pointer to NULL
58         tmp = stnode;
59     }
```

```
60 // Creating n nodes and adding them to the linked list
61 for(i = 2; i <= n; i++) {
62     fnNode = (struct node *)malloc(sizeof(struct node));
63
64     // Checking if memory allocation is successful
65     if(fnNode == NULL) {
66         printf(" Memory can not be allocated.");
67         break;
68     } else {
69         // Reading data for each node from user input
70         printf(" Input data for node %d : ", i);
71         scanf(" %d", &num);
72
73         fnNode->num = num; // Setting the data for fnNode
74         fnNode->nextptr = NULL; // Setting the next pointer to NULL
75
76         tmp->nextptr = fnNode; // Linking the current node to fnNode
77         tmp = tmp->nextptr; // Moving tmp to the next node
78     }
79 }
80 }
81 }
82
```

```
83 // Function to count the number of nodes in the linked list
84 int NodeCount() {
85     int ctr = 0;
86     struct node *tmp;
87     tmp = stnode;
88
89     // Counting the nodes by traversing the linked list
90     while(tmp != NULL) {
91         ctr++;
92         tmp = tmp->nextptr;
93     }
94     return ctr;
95 }
96
97 // Function to display the linked list
98 void displayList() {
99     struct node *tmp;
100     if(stnode == NULL) {
101         printf(" No data found in the list.");
102     } else {
103         tmp = stnode;
104         while(tmp != NULL) {
105             printf(" Data = %d\n", tmp->num);    // Printing the data of current node
106             tmp = tmp->nextptr;                  // Advancing the position of current node
107         }
108     }
109 }
```

4. Search Element Variants

- Write a program in C to search for an existing element in a singly linked list.

- *Test Data and Expected Output :*

- Input the number of nodes : 3

- Input data for node 1 : 2

- Input data for node 2 : 5

- Input data for node 3 : 8

-

- Data entered in the list are :

- Data = 2

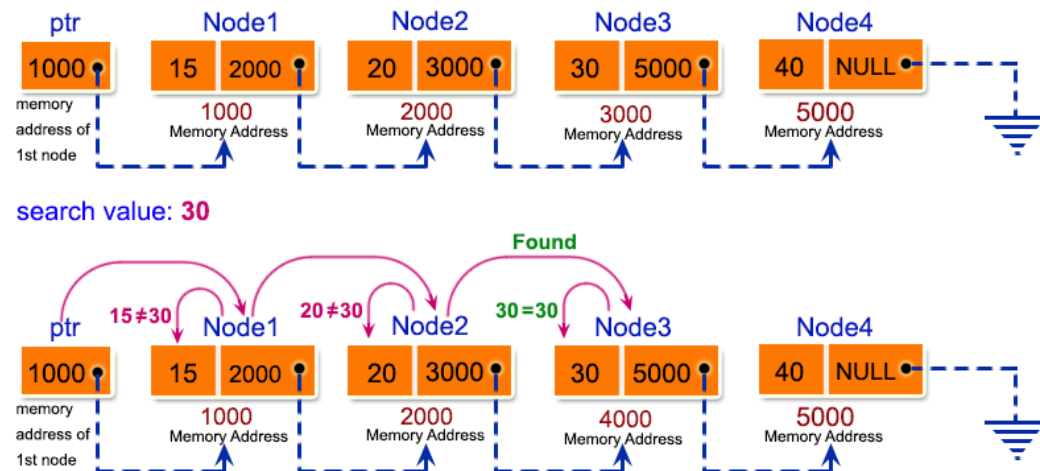
- Data = 5

- Data = 8

-

- Input the element to be searched : 5

- Element found at node 2



```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  // Define the structure for a node in a linked list
5  struct node
6  {
7      int num;           // Data of the node
8      struct node *nextptr; // Pointer to the next node
9  };
10
11 // Declare pointers to the starting node and a temporary node
12 struct node stnode, *ennode;
13
14 // Function to find an element in the linked list and return its position
15 int FindElement(int);
16
```

```
17 int main()
18 {
19     int n, i, FindElem, FindPlc;
20
21     // Initialize the starting node's pointer to NULL
22     stnode.nextptr = NULL;
23
24     // Point ennode to the starting node
25     ennode = &stnode;
26
27     printf("\n\n Linked List : Search an element in a Singly Linked List :\n");
28     printf("-----\n");
29
30     printf(" Input the number of nodes : ");
31     scanf("%d", &n);
32
33     printf("\n");
34     // Loop to create the linked list with user input data
35     for (i = 0; i < n; i++)
36     {
37         // Allocate memory for the next node
38         ennode->nextptr = (struct node *)malloc(sizeof(struct node));
39
40         printf(" Input data for node %d : ", i + 1);
41         scanf("%d", &ennode->num);
42
43         ennode = ennode->nextptr;
44     }
45     ennode->nextptr = NULL;
46
47     printf("\n Data entered in the list are :\n");
48
```

```
49 // Reset ennode to the starting node
50 ennode = &stnode;
51
52 // Loop to display the data entered in the linked list
53 while (ennode->nextptr != NULL)
54 {
55     printf(" Data = %d\n", ennode->num);
56     ennode = ennode->nextptr;
57 }
58
59 printf("\n");
60
61 printf(" Input the element to be searched : ");
62 scanf("%d", &FindElem);
63
64 // Call FindElement function to search for the element and get its position
65 FindPlc = FindElement(FindElem);
66
67 // Display the position of the found element or a message if it doesn't exist
68 if (FindPlc <= n)
69     printf(" Element found at node %d \n\n", FindPlc);
70 else
71     printf(" This element does not exist in the linked list.\n\n");
72 }
73
```

```
74 // Function to find an element in the linked list and return its position
75 int FindElement(int FindElem)
76 {
77     int ctr = 1;
78
79     // Reset ennode to the starting node
80     ennode = &stnode;
81
82     // Loop to traverse the linked list and find the element
83     while (ennode->nextptr != NULL)
84     {
85         // Check if the element is found at the current node
86         if (ennode->num == FindElem)
87             break;
88         else
89         {
90             ctr++;
91             ennode = ennode->nextptr;
92         }
93     }
94     return ctr; // Return the position of the found element
95 }
```